Kniholap Project Report: Audit & Optimization Guide

1. Executive Summary

Kniholap is a modern book marketplace and digital library platform built with **Next.js 16 (Canary/Future release)** and **React 19**. It features a robust integration with a Laravel backend, Stripe for payments, and standard modern tools like React Query, Framer Motion, and Tailwind CSS 4. The project is well-structured but has several areas where performance, maintainability, and security can be significantly improved.

---

2. Technical Audit

2.1 Architecture & Best Practices

- **Next.js App Router**: The project correctly utilizes the App Router, separating public routes `(main)` and `dashboard`.
- **Server Components**: Good use of Server Components for initial data fetching (e.g., in `(main)/page.js` and `library/book/[slug]/page.js`).
- **Data Fetching Consistency**: There is a mix of `axios` instances being created ad-hoc.
  - **Issue**: `axiosPrivateServer()`, `axiosPrivateClient()`, and `axiosPublic()` create new instances on every call.
  - **Impact**: Inefficient memory usage and potential issues with interceptors.

2.2 React Query Implementation

- **Issue**: In `app/providers.jsx`, the `QueryClient` is initialized inside the `Providers` component without memoization.
- **Impact**: The entire React Query cache is wiped and re-created whenever a parent component re-renders (though in this layout it's rare, it's still a critical anti-pattern).
- **Recommendation**: Initialize `QueryClient` outside the component or use a singleton pattern.

2.3 Styling & Design System

- **Issue**: Extensive use of `!important` in `globals.css` to override Ant Design styles.
- **Impact**: Makes the CSS brittle and hard to maintain. Themes are difficult to change.
- **Recommendation**: Utilize Ant Design's `ConfigProvider` for theme customization instead of CSS overrides where possible.

2.4 Security

- **Issue**: `sessionStorage` and console logs (Line 71 in `hooks/auth.hook.js`) store and log OTPs for development.
- **Impact**: Potential security risk if these are not strictly removed or guarded in production.
- **Issue**: The `NEXT_PUBLIC_BASE_URL` fallback is hardcoded in multiple places.

---

3. Improvement & Optimization Recommendations

3.1 Performance Upgrades

- **Dynamic Metadata**: Currently, most pages use static or templated metadata. For SEO, use `generateMetadata` in `[slug]` pages to fetch book-specific titles and descriptions.
- **Image Optimization**: While `next/image` is used, some images have fixed `width` and `height` that might not match the container's aspect ratio exactly, leading to layout shifts or sub-optimal loading.

- **Loading UI**: Enhance `loading.js` with more specific skeletons to improve perceived performance.

3.2 Code Quality & Maintainability

- **Centralize Axios Instances**: Create a single `api/index.js` or similar to export memoized or singleton instances.
- **Custom Hooks**: Refactor large logic blocks in components into smaller, reusable custom hooks.

---

4. Actionable Optimization Examples

Optimization Checklist & Snippets

A. Fix QueryClient Initialization

```
// app/providers.jsx
let client = null;
function getQueryClient() {
  if (!client) client = new QueryClient();
  return client;
}

export default function Providers({ children }) {
  const queryClient = getQueryClient();
  // ...
}
```

B. Clean up CSS Overrides

Instead of:

```
.ant-pagination .ant-pagination-item {
  border-radius: 50% !important;
}
```

Use:

```
// In a central config provider
<ConfigProvider theme={{
  token: {
    borderRadius: '50%',
  }
}}>
```

C. Secure Auth Flow

Remove development logs and use more secure ways to handle temporary tokens.

---

5. Security & SEO Audit Summary

Feature	Status	Priority	Recommendation

SEO	🟡 Needs Improvement	Medium	Implement `generateMetadata` for dynamic routes.
Performance	🟢 Good	Low	Optimize bundle by reviewing large dependencies like `swiper`.
Security	🟡 At Risk	High	Remove OTP logging and hardcoded dev URLs.
Maintainability	🔴 Needs Attention	High	Refactor CSS overrides and centralize API logic.

---

6. Next Steps

1. **Refactor Providers**: Move `QueryClient` initialization.
2. **Standardize API**: Centralize Axios instances.
3. **SEO Enhancement**: Add dynamic metadata to book and library pages.
4. **CSS Cleanup**: Move overrides to Ant Design theme configuration.